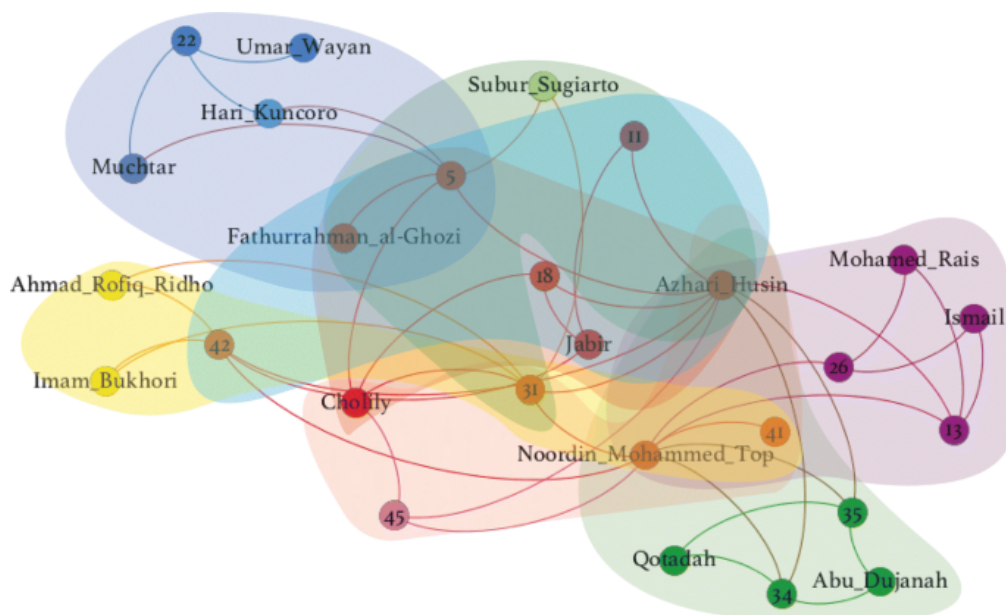


Projet d'outils pour la conception d'algorithmes

Détection de communautés dans des réseaux sociaux



Réaliser par :

-MALECOU NGUIMBI
Juvenal
-KAMDEM GUIAM
Valery Serena

Encadrant :

- George Manoussakis

IATIC4-2024/2025

CONTENTS

INTRODUCTION.....3

I. Partie 1 et 2 : Génération et Analyse de Graphes.....4

 1-Explications des structures de données4

 2-Méthodes , fonctions principales et complexité5

 3 -*Resultat*.....9

II. Partie 3 : Génération et Analyse de Graphes.....10

 1-fonctions principales et complexité.....10

 2-*Resultat*.....11

 CONCLUSION.....13

INTRODUCTION

. Un réseau social peut facilement être vu comme un graphe où les nœuds représentent les utilisateurs et les arêtes les liens qui existent entre eux dans le réseau. Le but de projet est d'étudier les méthodes de détection de communautés au sein de ces structures complexes, en se concentrant sur l'implémentation et l'analyse d'algorithmes de graphes. Ainsi nous avons identifié deux objectifs principaux : d'une part, développer une compréhension approfondie des algorithmes d'énumération de cliques et de bicliques maximales, et d'autre part, évaluer leur efficacité sur des graphes générés aléatoirement ainsi que sur des données réelles de réseaux sociaux. Cette démarche permettra non seulement d'acquérir des compétences pratiques en programmation et en analyse d'algorithmes, mais aussi de comprendre les enjeux théoriques et pratiques de la détection de communautés. Tout au long de ce projet, nous explorerons la génération de graphes, l'implémentation d'algorithmes complexes tels que Bron-Kerbosch, et l'analyse comparative de leurs performances en termes de temps d'exécution et d'utilisation de la mémoire.

I. Partie 1 et 2 : Génération et Analyse de Graphes

Pour tous le projet on va utiliser le langage python et comme librairie on utilisera principalement : **networkx,matplotlib,numpy et pandas**

1-Explications des structures de données

Pour ce projet les graphes seront représenté sous forme de dictionnaire pour stocker les nœuds et leurs arrêts ,ce qui permet une implémentation efficace des opérations de base qui seront énuméré plus tard avec une brève description dans la suite .

Ainsi la classe Graphe implémente une structure de graphe non orienté avec plusieurs méthode pour manipuler les nœuds et les arrêts don les fonctionnalité principale sont :

Rappel le graphe sont stocké comme liste d'adjacence choix qui me permettra d'utiliser quelques fonctions de la bibliothèque NetworkX bien que celle ci soit minime .Plus précisément on aura pour chaque graphe généré ou ceux de stanford donné comme suit :

- Le dictionnaire représentant le graphe
- la clé du dictionnaire représentant les nœuds
- et une liste de nœuds adjacents (voisins ainsi on s'assure de la définition complète d'un graphe

Ex : $G = \{ '0'; ['voisin_p0', \dots, 'voisin_d0'] , \dots, 'i'; ['voisin_pi', \dots, 'voisin_di'], \dots, \dots, 'n'; ['voisin_pn', \dots, 'voisin_dn'] \}$

ou l'intervalle entier $[0, n]$ designe les nœuds du graphe G ,

voisin_pi désigne le premier voisin pour le nœud i avec i appartenant a l'intervalle entier $[0, n]$

et voisin_di désigne le dernier voisin pour le nœud i avec i appartenant a l'intervalle entier $[0, n]$

2-Méthodes , fonctions principales et complexité

Classe graphe

➔ Méthode add_node(node) :

Temps : $O(1)$, car elle ajoute un nœud au dictionnaire ou vérifie simplement son existence.

Mémoire : $O(1)$ par nœud, puisque chaque nœud ajoute une clé unique avec une liste vide dans le dictionnaire.

➔ Méthode add_nodes_from(nodes) :

Temps : $O(n)$, avec n étant le nombre de nœuds dans nodes, car chaque appel à add_node prend $O(1)$.

Mémoire : $O(n)$ pour stocker les nouveaux nœuds ajoutés.

Méthode add_edges_from(edges) :

Temps : $O(m)$, avec m étant le nombre d'arêtes, chaque appel à add_edge prend $O(1)$.

Mémoire : $O(m)$ pour chaque arête ajoutée, chaque arête étant stockée deux fois (une fois pour chaque nœud).

➔ Méthode degree() :

Temps : $O(n)$, où n est le nombre de nœuds. Elle parcourt le dictionnaire des nœuds pour obtenir le nombre de voisins (degré) de chaque nœud.

Mémoire : $O(n)$ pour le dictionnaire des degrés de chaque nœud.

➔ Méthode complete_graph(nodes) :

Temps : $O(n^2)$, où n est le nombre de nœuds dans nodes, car chaque paire de nœuds nécessite un appel à add_edge.

Mémoire : $O(n^2)$ pour stocker toutes les arêtes d'un graphe complet.

➔ Méthode remove_edge(node1, node2) :

Temps : $O(1)$, car la suppression dans une liste prend $O(1)$ si l'élément est connu.

Mémoire : $O(1)$ par suppression d'arête.

➔ Méthode remove_edge_from(edges) :

Temps : $O(m)$, avec m le nombre d'arêtes dans edges, car chaque suppression est $O(1)$.

Mémoire : $O(1)$, cette méthode n'ajoute pas de nouvelle mémoire.

➔ Méthode nodes() :

Temps : $O(n)$, car elle parcourt les clés du dictionnaire.

Mémoire : $O(n)$ pour la liste des nœuds retournée.

➔ Méthode `edges()` :

Temps : $O(n+m)$, où n est le nombre de nœuds et m est le nombre d'arêtes. Chaque nœud et ses voisins sont parcourus pour identifier chaque arête une seule fois.

Mémoire : $O(m)$ pour la liste des arêtes uniques retournées.

➔ Méthode `add_edge(node1, node2)` :

Temps : $O(1)$, car l'ajout d'une arête à deux listes prend un temps constant.

Mémoire : $O(1)$ par ajout d'arête.

➔ Méthode `neighbors(node)` :

Temps : $O(1)$, car elle retourne simplement la liste des voisins d'un nœud.

Mémoire : $O(1)$, la liste retournée est simplement une référence.

➔ Méthode `has_edge(node1, node2)` :

Temps : $O(1)$, car elle vérifie l'appartenance à une liste.

Mémoire : $O(1)$, aucune mémoire supplémentaire n'est nécessaire.

➔ Méthode `draw()` :

Temps : $O(n+m)$, car elle convertit le graphe en un objet NetworkX puis utilise matplotlib pour dessiner.

Mémoire : $O(n+m)$ pour stocker temporairement le graphe dans NetworkX.

➔ Méthode `shortest_path_length(source, target)` :

Temps : $O(n+m)$, car elle utilise l'algorithme de recherche de plus court chemin de NetworkX (basé sur une BFS pour les graphes non pondérés).

Mémoire : $O(n)$ pour stocker les nœuds visités pendant la recherche.

➔ Méthode `from_pandas_edgelist(df, source, target)` :

Temps : $O(m)$, où m est le nombre d'arêtes dans `df`, car chaque arête est ajoutée.

Mémoire : $O(n+m)$, car elle ajoute n nœuds et m arêtes dans le graphe.

➔ Méthode `__str__()` :

Temps : $O(n+m)$, car elle convertit le dictionnaire des listes d'adjacence en une chaîne de caractères.

Mémoire : $O(n+m)$ pour la chaîne retournée.

✓ **Résumé Global**

Temps : La majorité des méthodes ont des complexités entre $O(1)$ et $O(n+m)$, sauf pour `complete_graph` qui est $O(n^2)$.

Mémoire : Principalement entre $O(1)$ et $O(n+m)$, avec le graphe complet ayant une complexité mémoire de $O(n^2)$.

Fonctions

➔ `generer_random_G(n, p, nb_G)` :

Complexité en Temps : $O(n^2 \cdot nb_G)$ car on parcourt toutes les paires de nœuds pour chacun des `nb_G` graphes.

Complexité en Mémoire : $O(n+e)$ pour chaque graphe, où `n` est le nombre de nœuds et `e` est le nombre attendu d'arêtes.

➔ `creer_graphe_depuis_csv(f)` :

Complexité en Temps : $O(m)$, où `m` est le nombre d'arêtes (lignes dans le fichier CSV).

Complexité en Mémoire : $O(n+m)$, où `n` et `m` sont le nombre de nœuds et d'arêtes respectivement.

➔ `creer_graphe_depuis_fichier(fichier)`

Complexité en Temps : $O(m)$, car chaque arête (ligne) du fichier est lue une fois.

Complexité en Mémoire : $O(n+m)$, où `n` est le nombre de nœuds et `m` est le nombre d'arêtes.

➔ `histo_G(Gs)` :

Complexité en Temps : $O(n+m)$ pour calculer les degrés et tracer les histogrammes pour chaque graphe dans la liste `Gs`.

Complexité en Mémoire : $O(n)$ pour stocker les degrés de chaque nœud.

➔ `histo_f(dossier)` ;

Complexité en Temps : $O(k \cdot (n+m))$, où `k` est le nombre de fichiers de graphes dans `dossier`, et `n` et `m` sont les nœuds et arêtes par graphe.

Complexité en Mémoire : $O(n)$ par graphe pour le stockage des degrés.

➔ `histo_csv(f)` :

Complexité en Temps : $O(n+m)$ pour calculer les degrés et tracer l'histogramme.

Complexité en Mémoire : $O(n)$ pour stocker les degrés.

➔ `nom_fichiers(dossier)` :

Complexité en Temps : $O(k)$, où `k` est le nombre de fichiers dans le dossier.

Complexité en Mémoire : $O(k)$ pour stocker la liste des noms de fichiers.

➔ `nb_chemins_induits(G)` :

Complexité en Temps : $O(n \cdot d^2)$, où n est le nombre de nœuds et d est le degré moyen des nœuds (car on vérifie toutes les paires de voisins pour chaque nœud).

Complexité en Mémoire : $O(1)$, car seule une variable de comptage est utilisée.

➔ `bron_kerbosch(R, P, X, graph, Liste_cliques_maximale=None)` :

Complexité en Temps : $O(3^n/3)$ dans le pire des cas, car chaque appel récursif peut avoir jusqu'à trois options pour chaque nœud.

Complexité en Mémoire : $O(n)$ pour la pile d'appels récursifs, plus l'espace pour stocker les cliques maximales trouvées.

➔ `bron_kerbosch_parallel(graph)` :

Complexité en Temps : Le temps dépend de la complexité de `bron_kerbosch` et de la parallélisation avec p processus, chacun prenant $O(3^n/3p)$.

Complexité en Mémoire : $O(p \cdot n)$ pour chaque processus et la pile récursive, plus l'espace pour fusionner les résultats des cliques maximales.

➔ `enumeration_cliques(Gs)`

Complexité en Temps : $O(t \cdot 3^n/3)$, où t est le nombre de graphes, chacun ayant une complexité maximale de $O(3^n/3)$.

Complexité en Mémoire : $O(n \cdot t)$ pour la pile d'appels, plus l'espace pour stocker toutes les cliques maximales.

➔ `complement_graph(G)` :

Complexité en Temps : $O(n^2)$ pour vérifier toutes les paires de nœuds et ajouter les arêtes manquantes.

Complexité en Mémoire : $O(n+e')$, où e' est le nombre d'arêtes dans le complément du graphe.

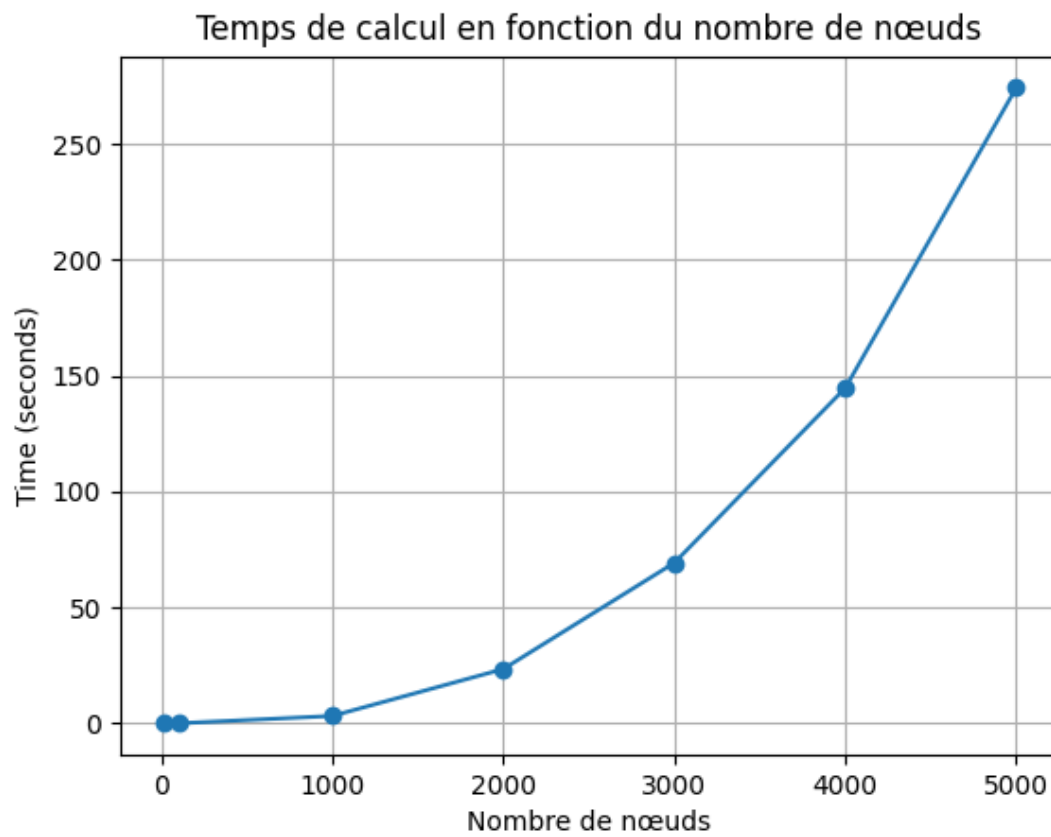
➔ `independant_maximal_G(G)` :

Complexité en Temps : Complexité de `complement_graph` plus celle de `bron_kerbosch`, soit $O(n^2 + 3^n/3)$

Complexité en Mémoire : $O(n+e)$ pour le complément du graphe plus l'espace pour les cliques maximales (équivalentes aux ensembles indépendants maximaux).

3 -Resultat

Titre : tableau de creation des graphes aléatoire avec $p=0.5$



Nom du graphe	Nombre chemins induits	Dégré Max	
g_no_fbk.txt	2184843	1045	
g_no_email.txt	510275	347	
g_no_LASN.csv	378335	216	

II. Partie 3 : Génération et Analyse de Graphes

1-fonctions principales et complexité

L'analyse des complexités en temps et en mémoire pour les fonctions de ce code, avec n le nombre de nœuds et m le nombre d'arêtes dans le graphe G .

➔ Fonction $V_i(i, G)$:

Complexité en temps : $O(n)$ La fonction crée une liste des nœuds de G , ce qui prend $O(n)$ temps, et parcourt les nœuds pour trouver l'indice de i .

Complexité en mémoire : $O(n)$ La liste temporaire V utilise $O(n)$ espace, car elle contient tous les nœuds de G .

➔ Fonction $N_k_v(i, k, G)$:

Complexité en temps : $O(n \cdot (n+m))$ Cette fonction appelle $V_i(i, G)$, qui est en $O(n)$, et parcourt chaque nœud de G pour calculer les plus courts chemins.

La complexité pour chaque calcul de plus court chemin est $O(n+m)$, donc parcourir tous les nœuds pour évaluer leur distance donne une complexité de $O(n \cdot (n+m))$.

Complexité en mémoire : $O(n)$ La liste N utilise $O(n)$ espace pour stocker les nœuds trouvés à distance k de i .

➔ Fonction $G_i(G, i)$:

Complexité en temps : Appels à $N_k_v(i, 1, G)$ et $N_k_v(i, 2, G)$, chacun ayant une complexité de $O(n \cdot (n+m))$.

Boucle for j in $[i] + N1 + N2$:

Ajoute les nœuds à G_i , soit $O(n)$.

Boucle for j in $G.edges()$:

Vérifie chaque arête de G pour ajouter les arêtes entre les nœuds à distance 1 ou 2. Cela prend $O(m)$ au total.

Double boucle for k in $N1$ et for j in $N2$:

La double boucle a une complexité au pire des cas en $O(\Delta \times (n - \Delta))$, où Δ est le degré maximal des nœuds dans $N1$.

Complexité totale : $O(n \cdot (n+m) + \Delta \times (n-\Delta))$.

Complexité en mémoire :

$O(n)$ pour stocker les nœuds $N1$ et $N2$ et $O(m)$ pour stocker les arêtes de G_i .

✓ Résumé global

Complexité en temps totale : $O(n \cdot (n+m) + \Delta \times (n-\Delta))$ ou encore $O(n \cdot (n+m) + n^2) = O(n \cdot (n+m) + n^2) = O(n \cdot (m+2 \cdot n))$ car $(n-\Delta) < n$

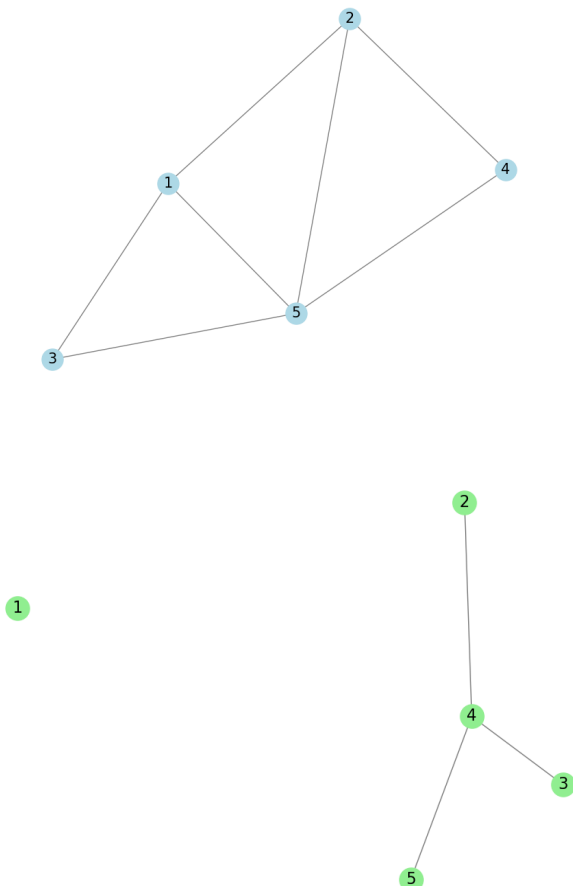
Complexité en mémoire totale : $O(n+m)$ pour le graphe partiel G_i .

2-Resultat

Les résultats seront présentés durant la présentation dans leur intégralité

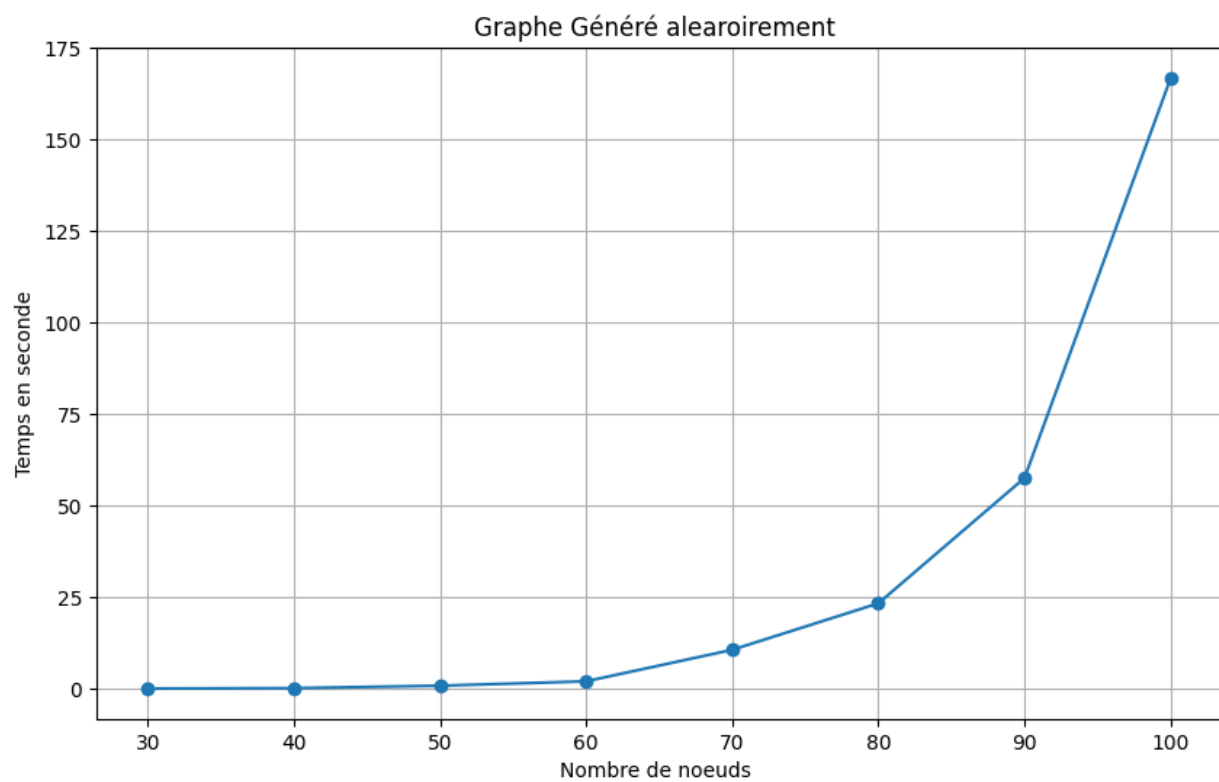
exemple d'image qui vont être présentées :

Graphe généré avec l'implémentation des l1 et l2 du papier



Graphe G_1 les autres seront présentés durant la présentation .

Clique maximale



CONCLUSION

Ce projet nous a permis d'explorer en profondeur plusieurs algorithmes de graphes et leurs implémentations. Nous avons développé et mis en œuvre des algorithmes sophistiqués tels que Bron-Kerbosch pour l'énumération de cliques et de bibliques maximales, démontrant ainsi notre compréhension approfondie de ces structures. Évaluer les performances des différents algorithmes analysés, les complexités en temps et en mémoire des différentes fonctions qui ont été implémentées .

Ce projet ouvre la voie à de futures recherches dans le domaine de l'analyse des réseaux sociaux, avec des applications potentielles dans divers domaines tels que le marketing ciblé ou l'étude de la propagation de l'information.

SOURCES

Pour l'algo de Bron-Kerbosh :: <https://www.sciencedirect.com/science/article/pii/S0304397508003903>

Pour l'implémentations des fonctionon a utiliser uniquement la documentation officielle de NetworkX :<https://networkx.org/documentation/stable/reference/functions.html>